Sincronização do Metrô

Implementação 4 - Sistemas Operacionais - 2026.2

CESAR School
Leonardo Gutzeit Borgmann Matos
Login: **lgbm** | E-mail: lgbm@cesar.school
Data: 04/11/2026

1. Objetivo

Implementar em C a sincronização de um vagão e de passageiros controlados por threads.
Garantir embarque dentro da capacidade, com todos os passageiros autorizados sentados antes da partida.
Permitir embarque simultâneo sem semáforos ou espera ocupada.
Entregar código, Makefile, evidências e este relatório, identificado por lgbm.

2. Checklist de Requisitos

Requisito do enunciado	Status	Como testar / evidência
Implementação em C com pthreads	OK	sync_car.c e sync_car.h; build remoto PASS informado pelo aluno.
Inicializar a estação e seus contadores	OK	station_init inicializa mutex, condições e três contadores.
Aguardar vagão e reservar vaga	OK	Runner verifica embarques esperados e capacidade em cada carro.
Partir apenas após todos sentarem	OK	boarding_passengers só diminui em station_on_board; runner verifica retorno prematuro.
Sair quando lotado ou sem fila	OK	Condição de station_load_car; runner inclui vagas acima da demanda.
Usar um único lock por estação	OK	Inspeção de struct station: um pthread_mutex_t.
Usar as sete funções pthread exigidas	OK	init/lock/unlock de mutex e init/signal/wait/broadcast de condição em sync_car.c.
Não usar semáforos ou outras primitivas	OK	Implementação usa somente mutex e variáveis de condição.
Permitir embarque simultâneo	OK	wait_for_car libera o lock antes de on_board; runner confirma embarques separadamente.
Não fazer espera ocupada	OK	Laços de espera da implementação chamam pthread_cond_wait.
Gerar executável sync_car com make	OK	Makefile da raiz gera ./sync_car no Linux; build remoto PASS.

3. Como Reproduzir

Ambiente de entrega: Linux, Unix ou macOS com compilador C, make e pthreads. No pacote, extraia a pasta lgbm e execute:

```
tar -xf lgbm.tar
cd lgbm
make clean
make
./sync_car
```

O runner não recebe argumentos. A saída deve terminar em **Looks good!**, com código de saída zero. As capacidades são aleatórias: os números de cada execução podem variar.

No repositório GitHub, o Makefile da raiz também gera o executável no local exigido pelo autocheck:

```
make clean
make
./sync_car
```

Para registrar uma nova sessão em Linux, inicie a captura antes de compilar:

```
script -a evidencias.log
date
whoami
pwd
make clean
make
./sync_car
exit
```

Validação efetivamente realizada neste trabalho: Windows com MinGW GCC 6.3.0, usando `mingw32-make -B test`. O log recente registra data, usuário, diretório, comando, saída e código de retorno. Não foi usada a ferramenta Unix script nesse ambiente. O runner desativa alarm() no Windows; as dez execuções iniciais tiveram limite externo de 15 segundos.

4. Arquitetura (resumida)

Arquivo	Responsabilidade
sync_car.h	Define struct station e declara as quatro funções da API.
sync_car.c	Controla fila, vagas e embarques pendentes com mutex e duas condições.
sync_car-runner.c	Fornece main e testes com 100 passageiros e vagas aleatórias.
Makefile	Compila código e runner; gera sync_car e permite executar testes.
evidencias.log	Reúne resultados locais e a transcrição identificada do resultado remoto.

Decisões técnicas

1. Um mutex por estação → proteger os três contadores em conjunto → reservas de vagas são atômicas.
2. Duas variáveis de condição → separar espera por vagas da espera pelo fim do embarque → passageiro e vagão aguardam eventos distintos.
3. Separar reserva de assento e confirmação → permitir vários passageiros embarcando → o vagão espera boarding_passengers chegar a zero.

5. Estratégias e Diário de Desenvolvimento

5.1 Estratégias da semana

Estratégia	Nome curto	Contexto	Motivo da troca
S1	Contadores e condições	Sincronizar fila, vagas e passageiros em embarque usando um mutex.	Sem troca de algoritmo; os ajustes seguintes foram na entrega e na compilação.

5.2 Diário de Tentativas

A ordem abaixo segue os blocos do evidencias.log. Os trechos antigos não possuem horário individual; por isso, a referência temporal usa a data da sessão e a ordem dos blocos, sem inventar horários. Os erros do autocheck foram colados no chat pelo aluno e não estavam gravados no log original.

#	Estrat.	O que tentei	Resultado	Hipótese/Causa	Quando	Evidência
1	S1	Executar o runner dez vezes.	Passou	-	04/11; bloco inicial	Log, linha 11; resumo 10/10.
2	S1	Acrescentar Makefile de entrada na raiz.	Local passou; remoto parcial	O Makefile delegado compilou na subpasta; autocheck buscava o binário na raiz.	04/11; após bloco inicial	Log, linha 147; erros remotos informados no chat.
3	S1	Isolar Unidade 2 na main e preservar trabalhos anteriores.	Local passou	-	04/11; bloco seguinte	Log, linha 154; branch codex/ja-produzido.
4	S1	Compilar diretamente para sync_car na raiz.	Local passou	-	04/11; após reorganização	Log, linha 159; commit 1f46ca6.
5	S1	Recompilar e testar para preparar o relatório.	Passou	-	04/11; sessão atual	Log, linha 166; Looks good! e saída 0.
6	S1	Registrar o resultado final do autocheck recebido.	3/3; 100%	-	04/11; recebimento após sessão atual	Log, linha 190; resultado fornecido pelo aluno.

O horário real do autocheck final não foi informado. A posição do registro no log indica quando o resultado foi recebido, e não quando o serviço o executou. Não houve alteração do algoritmo para resolver os erros de localização do Makefile, do executável e do PDF.

6. Evidências

6.1 Log automático

O arquivo **lgbm/evidencias.log** está incluído no pacote lgbm.tar. O trecho inicial reúne as dez execuções do runner; o trecho recente captura a validação usada para este relatório. Blocos intermediários são resumos das correções, não transcrições completas de terminal.

Prova	Localização e alcance
Dez execuções locais	Linha 11 até RESUMO: 10/10 execucoes aprovadas. Todas terminaram com Looks good!.
Validação recente	Linha 166. Comando mingw32-make -B test, saída do runner e código de saída zero.
Autocheck final	Linha 190. Transcrição resumida e identificada do resultado recebido do aluno: build PASS e três runners PASS.
Limites cobertos	Runner chama station_load_car com estação vazia e com zero vagas; também verifica capacidade e espera pelo embarque.
Caso inválido	Não testado. O enunciado não define comportamento para count negativo ou chamadas fora da sequência da API.

6.2 Prints

Pendente: anexar a captura do erro principal do autocheck (binário sync_car não encontrado) e a captura da validação final com 100% nos testes públicos. O aluno forneceu as saídas textuais no chat, mas ainda não enviou as imagens. Não foram produzidas imagens simulando capturas de terminal.

7. Uso de IA

Onde foi usada: Codex auxiliou na implementação e revisão da sincronização, na adaptação do Makefile, na execução dos testes locais, na organização do Git e na redação deste relatório. As alterações foram comparadas ao enunciado e às saídas registradas.

Prompts principais:
- Testar o arquivo adicionado e registrar os resultados em evidencias.log.
- Corrigir as falhas do autocheck e conferir o enunciado.
- Após cada envio, conferir comigo os códigos e verificar se atendem ao enunciado.
- Antes da implementação, procurar resquícios do identificador mla e substituir por lgbm.
- Simplificar os comentários do código.
- Criar o .tar atualizado e padronizar o relatório segundo o guia.

O prompt de limpeza expressa a orientação solicitada pelo aluno. A conferência registrada no chat ocorreu durante a revisão e não encontrou mla nos arquivos de texto da main; não há registro que comprove uma substituição anterior à implementação.

Validação manual do aluno: o aluno executou o autocheck e compartilhou os erros e o resultado final de 100%. Compilações, revisão estática e testes locais foram conduzidos com auxílio da IA; não são apresentados como testes manuais independentes do aluno.

8. Reflexão Final

A separação entre reservar uma vaga e confirmar o assento permite o embarque simultâneo.
O contador de embarques pendentes impede que o vagão saia antes de todos sentarem.
Os primeiros obstáculos do autocheck estavam na organização da entrega, não no algoritmo.
Um Makefile que compila com sucesso ainda pode gerar o executável no local errado.
Na próxima atividade, o teste inicial deve reproduzir a estrutura exigida pelo corretor.
Também é necessário registrar comandos, erros e horários desde a primeira sessão.
O resultado final informado pelo aluno foi de 100% nos três testes públicos.

9. Checklist Final de Entrega

Item	Confirmado?
Compilei do zero seguindo só o relatório	Parcial: reconstrução completa local com MinGW; sequência Linux de extração e make ainda não executada nesta sessão.
Rodei os testes e evidencias.log foi gerado	Sim: testes locais registrados e resultado remoto informado identificado no log.
Tenho prints obrigatórios	Pendente: faltam as duas capturas solicitadas no guia.
Testei pelo menos um caso limite e um inválido	Parcial: estação vazia e zero vagas cobertos; caso inválido não testado.
Preenchi a seção de uso de IA	Sim: seção 7.
Revisei o relatório e removi frases genéricas	Sim: decisões, resultados, fontes e pendências estão explícitos.
Nome e pacote da entrega	lgbm.pdf e lgbm.tar; log dentro da pasta lgbm no pacote. Enviar ambos na mesma atividade.

10. Se eu tivesse mais 2 horas

1. Extrair o pacote em uma pasta limpa no Linux e executar exatamente os comandos da seção 3.
2. Registrar essa sessão com script, date, whoami e pwd antes da compilação.
3. Acrescentar um teste com barreira para comprovar que todos reservam antes de sentar.
4. Testar duas estações independentes para verificar o isolamento dos contadores.
5. Documentar o contrato para count negativo e chamadas inválidas antes de criar testes.
6. Remover o aviso de parâmetro não usado em uma cópia local do runner, preservando o original.
7. Guardar imediatamente os prints do erro principal e da aprovação final.
8. Conferir o pacote, os nomes e os caminhos do corretor antes de gastar uma tentativa.